StaySphere

Project Documentation

StaySphere is a full-stack Hostel & PG Management System

1. Introduction

StaySphere is a comprehensive digital solution designed to streamline the management of hostels, paying guest (PG) accommodations, and shared living spaces. The system replaces manual ledgers and fragmented communication tools with a centralized, web-based platform.

The system provides dual interfaces: an **Admin/Manager Portal** for property owners to oversee operations, and a **Tenant Portal** for residents to view their dues, log complaints, and check notices.

2. System Architecture

The application follows a standard Client-Server architecture utilizing the MERN stack (MongoDB, Express.js, React/Vanilla JS, Node.js).

Frontend (Client-Side)

- **Technology:** HTML5, CSS3, Vanilla JavaScript.
- **Structure:** Separate views for Administrators and Tenants (e.g., `dashboard.html` vs `tenant-dashboard.html`).
- **Styling:** Modular CSS files for each functional domain (e.g., `agreements.css`, `maintenance.css`).
- **API Integration:** Asynchronous JavaScript (Fetch API) is used to communicate with the backend REST endpoints.

Backend (Server-Side)

- **Environment:** Node.js
- **Framework:** Express.js
- **Authentication:** JSON Web Tokens (JWT) for secure route protection, and bcrypt for password hashing.
- **Structure:** MVC-inspired architecture separating Routes, Controllers, and Models.

Database Layer

- **Technology:** MongoDB (NoSQL)

- **ODM:** Mongoose
- **Key Collections:** Users, Tenants, Rooms, MaintenanceRequests, Notices, Expenses, Rent, Visitors, Agreements, Attendance.

3. Core Modules & Features

3.1 User Authentication & Authorization

Secure login and registration system. Users are categorized by roles (e.g., Admin, Tenant). Passwords are encrypted before storage, and session state is managed via secure JWT tokens.

3.2 Room & Bed Management

Admins can add new rooms, define capacity (number of beds), set pricing, and monitor real-time occupancy status. The system prevents overbooking and tracks available slots.

3.3 Tenant Lifecycle Management

End-to-end tracking of tenants. Includes capturing personal details, assigning them to specific rooms/beds, managing digital agreements, and handling the move-out process.

3.4 Financial Operations

- **Rent Collection:** Automated tracking of monthly rent dues, payment status (Paid, Pending, Partial), and historical rent records.
- **Expense Tracking:** Module for management to log property expenses (electricity, water, repairs, salaries) to calculate overall profitability.

3.5 Maintenance & Ticketing

Tenants can log maintenance requests (e.g., "AC not working", "Plumbing issue") via their portal. Admins view a consolidated list of tickets and can update their status (Pending, In Progress, Resolved).

3.6 Communication & Security

- **Notices:** Digital bulletin board where admins can broadcast announcements to all tenants.

- **Visitor Logs:** Keep track of guests visiting the property, recording check-in/check-out times for enhanced security.
- **Attendance:** Track tenant presence, crucial for student hostels with curfews.

4. API Reference

The backend exposes a RESTful API. Below is a summary of the core endpoints mapped to their respective controllers.

Authentication (`/api/auth`)

Method	Endpoint	Description
POST	`/register`	Registers a new user/tenant in the system.
POST	`/login`	Authenticates user and returns a JWT token.
GET	`/me`	Fetches profile of the currently logged-in user.

Rooms (`/api/rooms`)

Method	Endpoint	Description
GET	`/`	Retrieves all rooms and occupancy stats.
POST	`/`	Creates a new room configuration.
PUT	`/:id`	Updates details of a specific room.
DELETE	`/:id`	Removes a room from the property.

Tenants (`/api/tenants`)

Method	Endpoint	Description
GET	`/`	List all active and past tenants.

POST	`/`	Onboard a new tenant and assign room.
GET	`/:id`	Retrieve comprehensive profile of a tenant.

Maintenance (`/api/maintenance`)

Method	Endpoint	Description
GET	`/`	Fetch all maintenance requests (Admin) or user-specific requests (Tenant).
POST	`/`	Submit a new maintenance ticket.
PUT	`/:id/status`	Update the status of a ticket (e.g., to Resolved).

Note: Similar CRUD structures apply for `/api/rent`, `/api/expenses`, `/api/notices`, `/api/visitors`, `/api/attendance`, *and* `/api/agreements`.

5. Setup & Installation Guide

Prerequisites

- Node.js (v14.x or higher recommended)
- MongoDB (Local instance or MongoDB Atlas cluster)
- Git

Backend Setup

1. Navigate to the backend directory:

```
cd backend
```

2. Install dependencies:

```
npm install
```

3. Configure Environment Variables. Create a `.env` file based on `.env.example`:

```
PORT=5000
MONGO_URI=your_mongodb_connection_string
JWT_SECRET=your_super_secret_key
```

4. Start the server:

```
npm start
# Server will run on http://localhost:5000
```

Frontend Setup

The frontend consists of static HTML, CSS, and JS files. You can serve them using any static file server or simply open `index.html` in a modern web browser to access the landing page.

```
cd frontend
# Using Python's built-in server as an example:
python -m http.server 8000
# Access via http://localhost:8000
```